

TicTacToc



P1: 2

P2: 2

P1

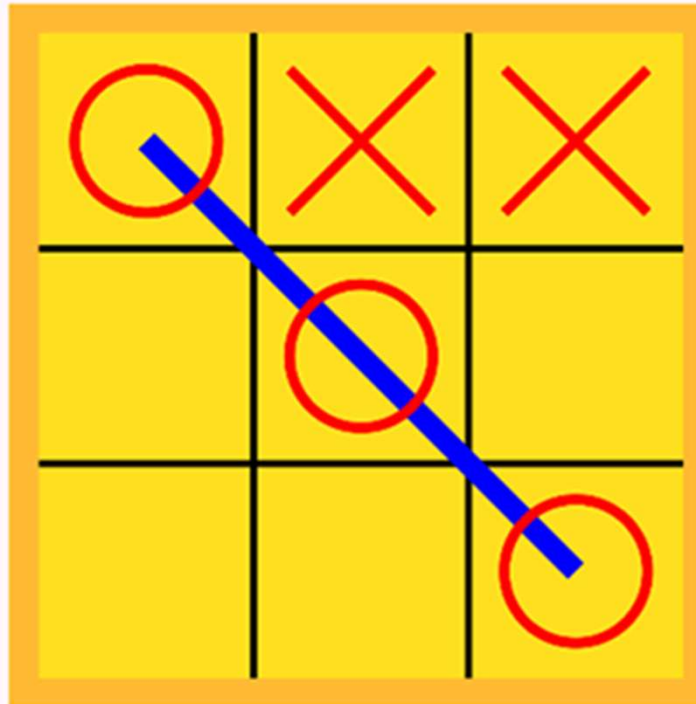
2025-10-17 10:35:44, Moves:7

P1

2025-10-17 10:36:01, Moves:8

P2

2025-10-17 10:36:10, Moves:5



New Game

Activity, Custom view and ViewModel

- MainActivity initializes the UI and the ViewModel, listens to user events and ViewModel events.
- The TicTacToeView draws the board and detects user touches, informing the listener.
- GameViewModel contains MutableLiveData for the game state and provides methods to start the game and process moves (to be implemented today).

Model

- This class encapsulates the state of the game, player turns, win tracking, and board state.
- The public API allows starting/restarting the game, making moves, and querying the board.
- The winner is determined by checking all rows, columns, and diagonals.
- Moves alternate between two players until there is a winner or a draw.
- Designed for perfect integration with UI/control

MVVM (Model-View-ViewModel)

ViewLayer

ViewModel

Model

MainActivity CustomView

- Acts as a **controller** or entry point managing the lifecycle of the **UI**
- **Observes** data exposed by the ViewModel and updates the UI accordingly.
- Rendering UI and **passing user events** to the ViewModel.

- **Bridge** between the **UI** and the **data layers**.
- Holds UI-related data and business logic in a lifecycle-conscious way.
- Exposes observable data (LiveData) that the View layer subscribes to for UI updates.
- Does not hold a reference to the UI components to avoid memory leaks.
- Requests or updates data by communicating with the Model.

- **Data** and **business rules**.
- **Abstracts** data sources such as databases, APIs, or repositories
- The Model is **oblivious** to UI and handles data operations
- The ViewModel interacts with the Model to **fetch**, **observe**, or **update** data

Observe data
passing user events

Fetch
observe
update data

MainActivity

```

MainActivity
  binding: ActivityMainBinding
  onCreate(savedInstanceState: Bundle?)
  styleScoreText(textViewOn: TextView, textViewOff: TextView)
  tttView: TicTacToeView
  viewModel: GameViewModel
  
```

CustomView

```

TicTacToeView
  OnPlayListener
    play(row: Int, col: Int)
  <class initializer>
  _onPlay: OnPlayListener?
  boardStates: Array<Array<TicTacToeModel.Players>>
  cellSize
  onDraw(canvas: Canvas)
  onPlay: OnPlayListener?
  onTouchEvent(event: MotionEvent?): Boolean
  paint
  setBoard(board: Array<Array<TicTacToeModel.Players>>)
  setOnPlayListener(callback: OnPlayListener)
  TicTacToeView(context: Context)
  
```

```
private val viewModel: GameViewModel by viewModels()
```

```
viewModel.winsP1.observe(this) {binding.player1Score.text=it }
```

ViewModel

```

GameViewModel
  board
  cur_player
  game
  gameFinished
  play(y: Int, x: Int)
  start()
  winner
  winsP1
  winsP2
  
```

```
val game = TicTacToeModel()
```

```
val winsP1 = MutableLiveData<String>()
```

```
winsP1.value = game.winsP1.toString()
```

Model

```

TicTacToeModel
  Players
    None
    P1
    P2
  <class initializer>
  _ctrl
  _currentPlayer
  _gameFinished
  _winsP1
  _winsP2
  board: Array<Array<Players>>
  checkForWinner(): Players
  currentPlayer: Players
  gameFinished: Boolean
  get(y: Int, x: Int): Players
  getBoard(): Array<Array<Players>>
  play(y: Int, x: Int): Boolean
  reset()
  start()
  Winner: Players
  winsP1: Int
  winsP2: Int
  
```

```

public val winsP1 : Int
get() = _winsP1
  
```

To do

Complete ViewModel Logic

- Implement the `start()` method in `GameViewModel` to reset the game, update `LiveData` for board state and resets scores as needed.
- Implement the `play(y, x)` method to:
 - Call the `TicTacToeModel.play(y, x)` to register the move.
 - Update the board `LiveData` with the new board state.
 - Update `winsP1` and `winsP2` `LiveData` based on the model's win counts.
 - Notify UI of game end (e.g., disable board input if finished).

Update TicTacToeView to Draw Player Moves

- Extend the `onDraw()` method in `TicTacToeView` to draw X or O marks inside each cell based on the current board state from `ViewModel`.
- Create a method in `TicTacToeView` to accept the board state and trigger redraw when the board updates.

Connect ViewModel's Board LiveData to View

- In `MainActivity`, observe the board `LiveData`.
- When board updates occur, send the updated board to `tttView` to redraw the marks.

Complete ViewModel Logic: start()

- Implement the `start()` method in `GameViewModel` to:
 - reset the game
 - update LiveData for board state and resets scores as needed.

```
fun start() {  
    game.start() // Reset model  
    winsP1.value = game.winsP1.toString()  
    winsP2.value = game.winsP2.toString()  
    board.value = game.getBoard()  
}
```

Complete ViewModel Logic: play

- Implement the `play(y, x)` method to:
 - Call the `TicTacToeModel.play(y, x)` to register the move.
 - Update the `board LiveData` with the new board state.
 - Update `winsP1` and `winsP2 LiveData` based on the model's win counts.
 - Notify UI of game end (e.g., disable board input if finished).

```
fun play(y: Int, x: Int) {  
    if (game.play(y, x)) {  
        board.value = game.getBoard() // Update board LiveData  
        winsP1.value = game.winsP1.toString()  
        winsP2.value = game.winsP2.toString()  
    }  
}
```


Update **TicTacToeView** to Draw Player Moves

- Extend the `onDraw()` method in `TicTacToeView` to draw X or O marks inside each cell based on the current board state from `ViewModel`.
- Create a method in `TicTacToeView` to accept the board state and trigger redraw when the board updates.

Custom View

1. Add a property to hold the board state:

```
private var boardStates: Array<Array<TicTacToeModel.Players>> =  
    Array(3) { Array(3) { TicTacToeModel.Players.None } }  
  
fun setBoard(board: Array<Array<TicTacToeModel.Players>>) {  
    boardStates = board  
    invalidate() // Triggers onDraw()  
}
```

Update TicTacToeView to Draw Player Moves

2 - Modify onDraw():

```
override fun onDraw(canvas: Canvas) {
    super.onDraw(canvas)
    cellSize = width / 3f

    // Draw grid lines (existing code)
    for (i in 1..2) {
        canvas.drawLine(0f, cellSize * i, width.toFloat(), cellSize * i, paint)
        canvas.drawLine(cellSize * i, 0f, cellSize * i, height.toFloat(), paint)
    }

    // Draw X or O marks inside each cell
    val symbolPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.RED
        strokeWidth = 15f
        style = Paint.Style.STROKE
    }
    for (row in 0..2) {
        for (col in 0..2) {
            val cx = col * cellSize + cellSize / 2
            val cy = row * cellSize + cellSize / 2
            val radius = cellSize / 3

            when (boardStates[row][col]) {
                TicTacToeModel.Players.P1 -> { // Draw X
                    canvas.drawLine(cx - radius, cy - radius, cx + radius, cy + radius, symbolPaint)
                    canvas.drawLine(cx + radius, cy - radius, cx - radius, cy + radius, symbolPaint)
                }
                TicTacToeModel.Players.P2 -> { // Draw O
                    canvas.drawCircle(cx, cy, radius, symbolPaint)
                }
                else -> {} // None, do nothing
            }
        }
    }
}
```

Custom View

Connect ViewModel's Board LiveData to View

- In MainActivity, observe the board LiveData.
- When board updates occur, send the updated board to tttView to redraw the marks.

In onCreate():

Activity

```
viewModel.board.observe(this) {  
    board -> tttView.setBoard(board)  
}
```

Handle Game End and New Game Button

- When the ViewModel reports gameFinished, show a message to the user (Toast, Dialog). In onCreate():

```
viewModel.gameFinished.observe(this) {  
    if (it != true) return@observe  
    AlertDialog.Builder(this)  
        .setTitle("Game Result")  
        .setMessage(  
            when (viewModel.winner.value) {  
                TicTacToeModel.Players.None -> "Draw"  
                else -> "${viewModel.winner.value} wins"  
            }  
        )  
        .setPositiveButton("OK", null)  
        .show()  
}
```

Activity

ViewModel

```
val gameFinished = MutableLiveData<Boolean>()  
val winner = MutableLiveData<TicTacToeModel.Players>()  
  
...  
fun play(y : Int, x : Int) {  
    winner.value = game.Winner  
    gameFinished.value = game.gameFinished  
}
```

Model

```
private var _gameFinished = true  
public val gameFinished : Boolean  
    get() = _gameFinished  
  
public val Winner : Players  
    get() = checkForWinner()
```

- On start App, disable input on the board until user taps "New Game".
 - Disable call to start() method on init of the model
- On "New Game" button press, reset the game state by calling ViewModel's start().
 - Already made on last practical class

Highlight the current player

- Change TextColor and TextSize

P1: 0

P2: 0

```
viewModel.cur_player.observe(this) {  
    if (it==TicTacToeModel.Players.P1)  
        styleScoreText(binding.player1Score, binding.player2Score)  
    else  
        styleScoreText(binding.player2Score, binding.player1Score)  
}  
...  
fun styleScoreText(textViewOn: TextView, textViewOff: TextView) {  
    textViewOn.apply {  
        setTextColor(Color.RED)  
        set textSize(TypedValue.COMPLEX_UNIT_SP, 28f)  
    }  
    textViewOff.apply {  
        setTextColor(Color.BLACK)  
        set textSize(TypedValue.COMPLEX_UNIT_SP, 16f)  
    }  
}
```

Activity

```
val cur_player = MutableLiveData<TicTacToeModel.Players>()  
...  
fun start () {  
    cur_player.value= game.currentPlayer  
...  
fun play (y : Int, x : Int) {  
    cur_player.value= game.currentPlayer
```

ViewModel

```
private var _currentPlayer = Players.None  
public val currentPlayer : Players  
    get() = _currentPlayer
```

Model

Show winning line visually on the board

Model

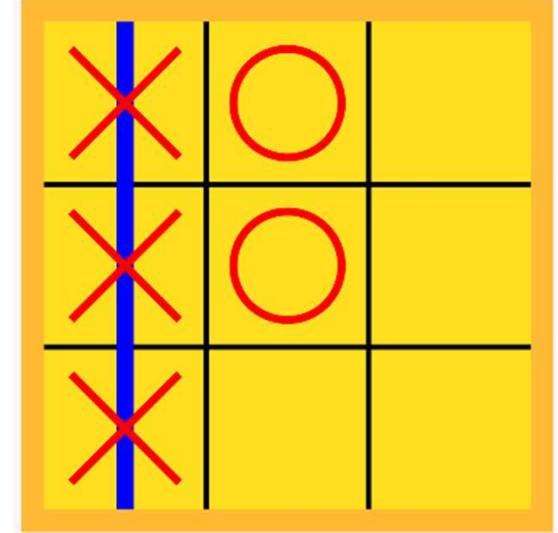
```
enum class WinType {
    None,
    ROW,
    COLUMN,
    DIAGONAL_MAIN,
    DIAGONAL_ANTI
}

data class WinLine(
    val type: WinType,
    val index: Int? = null // index can be null (diagonals)
)

private lateinit var _winLine :WinLine

public val winLine : WinLine
    get() = _winLine

...
public fun start() {
    _winLine= WinLine (WinType.None)
    ...
private fun checkForWinner(): Players {
    ...
    _winLine= WinLine(WinType.ROW, i) // On check rows, before return board[i][0]
    _winLine= WinLine(WinType.COLUMN, i) // On check columns, before return board[0][i]
    _winLine= WinLine(WinType.DIAGONAL_MAIN) // On check diagonal main, before return board[0][0]
    _winLine= WinLine(WinType.DIAGONAL_ANTI) // On check diagonal anti, before return board[0][2]
```



Show winning line visually on the board

Custom View

```
var winLine: TicTacToeModel.WinLine? = null
set(value) {
    field = value
    invalidate() // Redraw view to reflect changes
}
```

```
fun onDraw(canvas: Canvas) {
    winLine?.let { line ->
        val paint = Paint().apply {
            color = Color.BLUE
            strokeWidth = 32f
        }
        val index = line.index ?: -1
        when (line.type) {
            TicTacToeModel.WinType.None -> Unit
            TicTacToeModel.WinType.ROW -> {
                val y = cellSize * index + cellSize / 2
                canvas.drawLine(0f, y, width.toFloat(), y, paint)
            }
            TicTacToeModel.WinType.COLUMN -> {
                val x = cellSize * index + cellSize / 2
                canvas.drawLine(x, 0f, x, height.toFloat(), paint)
            }
            TicTacToeModel.WinType.DIAGONAL_MAIN -> {
                val offset = cellSize / 2
                canvas.drawLine(offset, offset, width.toFloat() - offset, width.toFloat() - offset, paint)
            }
            TicTacToeModel.WinType.DIAGONAL_ANTI -> {
                val offset = cellSize / 2
                canvas.drawLine(width.toFloat() - offset, offset, offset, width.toFloat() - offset, paint)
            }
        }
    }
}
```

Activity

```
viewModel.winLine.observe(this) { winLine ->
    tttView.winLine = winLine
}
```

Model View

```
val winLine = MutableLiveData<TicTacToeModel.WinLine>()
fun start() {
    winLine.value = game.winLine
}
fun play(y : Int, x : Int) {
    winLine.value = game.winLine
}
```

ListView : Scores history

- History:

- Player
- Date and Time
- Number of moves

Model

```
val list = ArrayList<Map<String, String>>()
fun getScoreList(): ArrayList<Map<String, String>> = list
```

```
// fun play ...
```

```
val sdf = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
val curDateTime = sdf.format(Date())
```

```
list.add(mapOf("Player" to "P1", "Score" to (curDateTime.toString() + ", Moves:" + (9 - _ctrl).toString())))
```

ViewModel

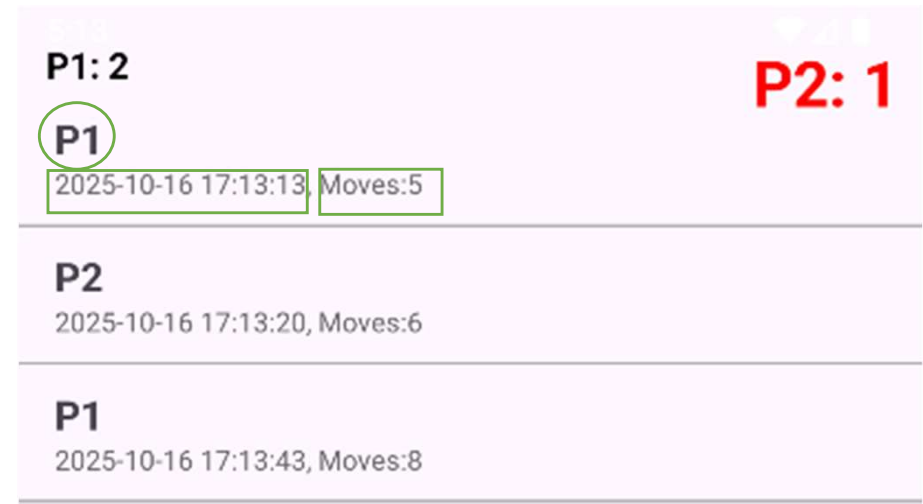
```
private val _scoreList = MutableLiveData<ArrayList<Map<String, String>>>()
val scoreList: LiveData<ArrayList<Map<String, String>>>
    get() = _scoreList
```

```
// fun start ...
```

```
_scoreList.value = game.getScoreList()
```

Activity

```
viewModel.scoreList.observe(this) { list ->
    val from = arrayOf("Player", "Score")
    val to = intArrayOf(R.id.tvPlayer, R.id.tvScore)
    val adapter = SimpleAdapter(this, list, R.layout.list_item, from, to)
    binding.listView.adapter = adapter
}
```



main_activity.xml

```
<ListView
    android:id="@+id/listView" />
```

list_item.xml

```
<TextView
    android:id="@+id/tvPlayer"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textStyle="bold"
    android:textSize="18sp" />
```

```
<TextView
    android:id="@+id/tvScore"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="12sp"
    android:textColor="#666" />
```